

Hito 3



Alejandro Peña Fernández

Índice:

RESUMEN:	4
Sistema de gestión para comunidades de vecinos:	4
Acceso al sistema:	4
Módulo de presupuesto:	4
Fondo de emergencia:	4
Módulo de gastos:	4
Tablón de votaciones:	5
Tecnologías:	5
DESCRIPCIÓN DETALLADA:	5
COMPARTIDOS:	5
NAVBAR:	5
FOOTER:	5
ESPECÍFICOS:	6
INICIO:	6
PRESUPUESTOS:	6
GASTOS:	6
VOTACIÓN:	6
OBJETIVOS:	7
Objetivo principal:	7
Objetivos específicos:	7
- Qué entra (alcance del MVP):	7
- Qué no entra (límites):	8
- Soluciones a posibles problemas:	8
CONTENIDOS DEL CICLO:	8
Desarrollo web en entorno servidor (DWES):	8
Desarrollo web en entorno cliente (DWEC):	8
Diseño de interfaces web (DIW):	8
ENTORNO DE TRABAJO:	9
PLANIFICACIÓN PRELIMINAR:	9
- Sprint 1:	9
- Sprint 2 (Login):	10
- Sprint 3 (Pisos):	10
- Sprint 4 (Usuarios):	10
- Sprint 5 (Home):	10
- Sprint 6 (Registro):	10
- Sprint 7 (Presupuesto y gastos):	11
- Sprint 8 (Votaciones):	11
- Sprint 9 (Despliegue):	11
- Sprint 10 (Comprobación):	11
HITO 3:	12
Identidad visual y Prototipo:	12

- Navbar y footer:	12
- Selección de comunidad:	13
- Login:	13
- Registro:	14
- Home:	15
- Usuarios:	15
- Generación de invitación:	16
- Pisos:	16
- Presupuesto:	17
- Historial de presupuestos:	17
- Gastos:	18
- Votaciones:	18
- Páginas que no existen o accesos no autorizados:	19
Base de datos:	19
Manual de uso:	20
- Primero clonamos el repositorio:	20
- Agregamos las variables de entorno:	20
- Levantar la base de datos:	21
- Arrancar el backend:	21
- Arrancar el frontend:	21
- Abrir la aplicación web:	21
- Uso:	21
- Borrar la base de datos:	21
Lecciones aprendidas:	22
- El hash de contraseñas en los datos iniciales	22
- Records en Java para DTOs	22
- JwtService manual vs Spring Security OAuth2 Resource Server	22
- Seguridad con Boolean.FALSE.equals() para evitar NullPointerException	22
- Separación de DTOs en request y response	23
- Mala estimación de tiempos y replanificación	23

RESUMEN:

Sistema de gestión para comunidades de vecinos:

Software diseñado para facilitar la administración de comunidades de vecinos, permitiendo al presidente gestionar presupuestos, gastos y decisiones comunitarias, mientras que los vecinos pueden consultar la información de manera transparente y ejercer su derecho a voto. El objetivo es agilizar los procesos administrativos y optimizar el tiempo de todos los miembros de la comunidad.

Acceso al sistema:

El acceso a la plataforma se realizará mediante un sistema de inicio de sesión. El registro de nuevos usuarios será responsabilidad del presidente de la comunidad cuando se incorpore un nuevo vecino al edificio.

Módulo de presupuesto:

Esta sección mostrará el presupuesto anual acordado por la comunidad, el detalle de gastos ejecutados y el saldo disponible tras descontar los gastos actuales. El presidente será responsable de registrar el presupuesto anual y los gastos mensuales.

Fondo de emergencia:

Se habilitará una pestaña específica donde se visualizará el fondo de emergencia, que vendrá del dinero sobrante al finalizar cada año.

Módulo de gastos:

Esta sección presentará una lista detallada con los gastos fijos mensuales y los gastos variables del mes en curso. El presidente podrá añadir nuevos gastos mediante un botón específico. Por defecto, los gastos se descontarán del presupuesto anual.

Tablón de votaciones:

Se dispondrá de un área donde se mostrarán las votaciones en curso y aquellas finalizadas en el último mes. La información se presentará en forma de lista con el título de cada votación y un botón para acceder a información detallada.

Para votaciones en curso donde el usuario no haya participado, se abrirá una ventana que permitirá votar a favor, en contra o abstenerse. Una vez emitido el voto, estos botones quedarán bloqueados.

En el caso de votaciones finalizadas, se mostrará la misma información, pero en lugar de los botones de votación aparecerá el resultado: aprobada o denegada.

Las votaciones concluirán cuando todos los vecinos hayan ejercido su voto o al alcanzarse la fecha límite establecida.

Tecnologías:

Las tecnologías a utilizar son HTML, CSS, Javascript, React, Java, Spring Boot y PostgreSQL.

DESCRIPCIÓN DETALLADA:

COMPARTIDOS:

NAVBAR:

Tendrá presupuestos, gastos y votaciones; que llevará respectivamente a sus páginas correspondientes.

FOOTER:

Tendrá el nombre del bloque con el símbolo de copyright y el año actual. Excepto en la página de inicio, tendrá un botón volver que llevará a la página de inicio.

ESPECÍFICOS:

INICIO:

Aparecerá una foto del bloque o algún fondo chulo y la bienvenida al edificio concreto.

PRESUPUESTOS:

Tendrá el gastos pactado anual, los gastos anuales que llevan hechos y el dinero actual que quede. Estos valores por defecto vendrán en 0 hasta que el presidente los cambie, para ello habrá un botón editar que llevará a un formulario en el que se pondrá el presupuesto anual y los gastos fijos de ese año con su respectivo nombre y descripción. El gastos que se lleva y el dinero actual será un cálculo del presupuesto anual junto con los gastos mensuales.

También se verá una lista de los gastos fijos anuales con los gastos en años y meses.

Abajo del todo se verá el fondo de emergencia que será el dinero guardado anualmente del sobrante del presupuesto anual.

GASTOS:

Se diferenciarán dos tarjetas, una con una lista de gastos fijos y otra con una lista de gastos imprevistos mensuales.

VOTACIÓN:

Una página con una lista de votaciones tanto pasadas como las presentes con la opción de entrar en cada una.

En las votaciones verás la información del motivo de la votación y podrás votar a favor, en contra o abstenerse, una vez votado no podrás volver a votar.

En votaciones acabas verás la información de la votación y cuál fue el resultado.

OBJETIVOS:

Objetivo principal:

Crear una aplicación web para que las comunidades de vecinos puedan gestionar su día a día sin tener que depender de papeles, grupos de WhatsApp o reuniones eternas. La idea es que el presidente pueda llevar los números y las votaciones desde la plataforma, y que los vecinos puedan ver en qué se gasta el dinero y votar cuando toque.

Objetivos específicos:

Sistema de acceso con roles: Que cada usuario entre con su cuenta y vea lo que le corresponde. El presidente puede hacer cosas que un vecino normal no (como registrar gastos o crear votaciones). Los vecinos nuevos los da de alta el presidente.

Gestión de presupuestos: Que el presidente pueda meter el presupuesto anual y los gastos fijos del año. Que se calcule automáticamente cuánto se lleva gastado y cuánto queda. Que se vea el fondo de emergencia con lo que sobre de cada año.

Control de gastos: Que se puedan ver los gastos fijos y los imprevistos del mes por separado. Que el presidente pueda añadir gastos nuevos y que se descuenten solos del presupuesto.

Tablón de votaciones: Que los vecinos puedan votar a favor, en contra o abstenerse en las votaciones abiertas. Que una vez votes no puedas cambiar el voto. Que las votaciones se cierren cuando todos hayan votado o cuando se pase la fecha límite. Que se vea el resultado de las votaciones que ya han acabado.

- Qué entra (alcance del MVP):

Un solo edificio/comunidad por instalación.

Dos roles: presidente y vecino.

CRUD de presupuestos, gastos y votaciones.

Autenticación con usuario y contraseña.

Consulta de información financiera por parte de los vecinos.

Votaciones con tres opciones (a favor, en contra, abstención).

- Qué no entra (límites):

No se gestionan cobros reales, ni pasarelas de pago. Esto solo refleja números, no mueve dinero.

No hay chat, ni mensajería entre vecinos.

No se gestiona la morosidad ni los recibos individuales de cada vecino.

No hay notificaciones por email ni push de momento.

No se contempla que haya varios edificios en la misma plataforma (multitenancy).

No se implementa recuperación de contraseña en el MVP.

No hay subida de documentos (actas, facturas escaneadas...).

- Soluciones a posibles problemas:

En un futuro se ampliará un apartado en el que se cometen diferentes soluciones o propuestas a problemas legales relacionados con la normativa española.

CONTENIDOS DEL CICLO:

Desarrollo web en entorno servidor (DWES):

Java y Spring Boot para el backend creando una API Rest para la lógica de negocio, autenticar al usuario con sus diferentes roles...

Desarrollo web en entorno cliente (DWECE):

Javascript y React para la interfaz visual creando componentes para la votación, visualización de presupuestos, navegar entre pestañas...

Diseño de interfaces web (DIW):

CSS para la estética visual de la aplicación web diseñando una interfaz sencilla para el usuario, con armonía y que entre por los ojos.

Despliegue de aplicaciones web:

Se usará algún proveedor, todavía no definido, para que se despliegue la aplicación y se conecten las diferentes partes.

ENTORNO DE TRABAJO:

Para el desarrollo del proyecto utilizaremos una carpeta principal que contendrá todo el código y la documentación, organizada de la siguiente manera: frontend, backend, docs, docker (si aplica según decisiones a tomar más adelante), .gitignore y README.md.

La carpeta docs/ albergará la bitácora del proyecto, que por ahora se llevará en un documento Word con el seguimiento de decisiones y lecciones aprendidas a lo largo del desarrollo. Más adelante se decidirá el formato concreto de entrega.

El .gitignore estará configurado desde el inicio para excluir archivos generados automáticamente como node_modules, los archivos de variables de entorno y otras configuraciones sensibles.

Para el control de versiones utilizaremos Git con tres ramas principales: main para el código estable, developer como rama de integración y feature/nombre-funcionalidad para el desarrollo de cada nueva funcionalidad. Además, contaremos con un repositorio remoto en GitHub.

Respecto al despliegue, el proyecto está pensado para funcionar sobre Vercel (frontend), Render (backend) y Neon (base de datos PostgreSQL), aunque también se contempla la opción de despliegue en una VPS o algún servicio con mejores prestaciones en el momento como alternativa.

PLANIFICACIÓN PRELIMINAR:

- Sprint 1:

Investigar tecnologías y definir stack -> 1 hora y 45 minutos

Diseñar wireframes y prototipo de la aplicación → 2 horas y 30 minutos

Crear esquema de base de datos y definir entidades/relaciones → 2 horas y 30 minutos

Preparar infraestructura (Docker, estructura de carpetas, Git...) → 1 hora y 45 minutos

Crear endpoints iniciales -> 30 minutos

- Sprint 2 (Login):

Backend: Autenticación, JWT → 3 horas y 30 minutos

Frontend: Formulario, protección de rutas...→ 2 horas y 45 minutos

Pruebas y correcciones -> 1 hora y 30 minutos

Estudiar nuevos conocimientos -> 1 hora

- Sprint 3 (Pisos):

Backend: Endpoints crear, listar, editar... -> 3 horas

Frontend: Página listado y acciones -> 3 horas

Pruebas y correcciones -> 1 hora

Estudiar nuevos conocimientos -> 1 hora

- Sprint 4 (Usuarios):

Backend: Endpoints listar, editar usuarios... -> 3 horas

Frontend: Página listado y acciones -> 3 horas

Pruebas y correcciones -> 1 hora

Estudiar nuevos conocimientos -> 1 hora

- Sprint 5 (Home):

Backend: Endpoints datos del usuario logeado y resumen comunidad -> 2 horas

Frontend: Dashboard inicial y navbar -> 3 horas

Pruebas y correcciones -> 1 hora

Estudiar nuevos conocimientos -> 1 hora

- Sprint 6 (Registro):

Backend: Generar código, validar código, registro con invitación... -> 4 horas

Frontend: Mostrar código invitación y página registro -> 4 horas

Pruebas y correcciones -> 2 horas

Estudiar nuevos conocimientos -> 1 hora

- Sprint 7 (Presupuesto y gastos):

Backend: CRUD presupuesto y CRUD gastos → 4 horas

Frontend: Presupuesto actual, historial... → 4 horas

Pruebas y correcciones -> 2 horas

Estudiar nuevos conocimientos -> 1 hora

- Sprint 8 (Votaciones):

Backend: Crear votación, votar, cierre... -> 4 horas

Frontend: Listado, detalles, formulario de voto... -> 4 horas

Pruebas y correcciones -> 2 horas

Estudiar nuevos conocimientos -> 1 hora

- Sprint 9 (Despliegue):

Desplegar y configurar base de datos -> 1 hora

Desplegar backend -> 2 horas

Desplegar frontend -> 1 hora

Probar y solucionar problemas -> 1 hora

Estudiar nuevos conocimientos -> 1 hora

- Sprint 10 (Comprobación):

Revisión de diseño y validación externa -> 1 hora y 30 minutos

Probar todos los flujos -> 2 horas

Corrección de fallos -> 1 hora y 30 minutos

Documentación técnica final -> 1 hora

Estudiar nuevos conocimientos -> 1 hora

HITO 3:

Identidad visual y Prototipo:

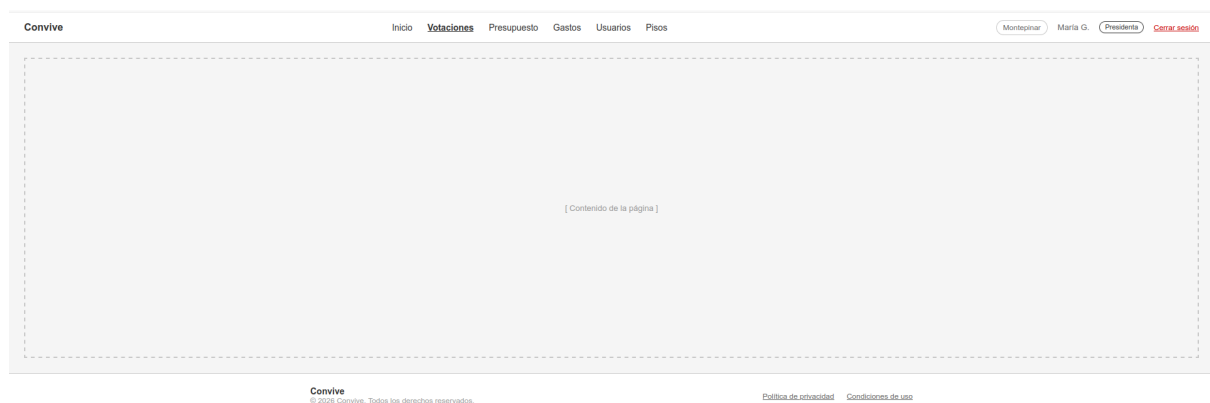
La identidad visual de Convive busca transmitir cercanía y sencillez. El público objetivo son vecinos de todas las edades, incluyendo personas mayores que pueden no estar familiarizadas con aplicaciones web. Por eso el diseño apuesta por una estética limpia y sin distracciones:

Fondo claro en rosa suave (#FDF6F6), un color principal cálido (#F7CAC9) que evoca hogar y comunidad, y un azul grisáceo de contraste (#656E95) para botones y elementos de acción que aporta seriedad sin resultar frío.

Se evitan deliberadamente los diseños recargados, los iconos excesivos y las animaciones innecesarias. Los formularios son directos, los textos grandes y legibles, y la navegación es intuitiva. La idea es que un vecino de 70 años pueda entrar, votar en una propuesta y salir sin necesitar ayuda, y que al mismo tiempo un presidente joven sienta que está usando una herramienta profesional y moderna.

A Continuación tenemos los wireframes de las páginas que conformarán mi aplicación teniendo una visión completa del alcance que se quiere lograr.

- Navbar y footer:



- Selección de comunidad:

Convive

Selecciona tu comunidad para continuar

Buscar comunidad...

Montepinar
Calle Girasol 12, Sevilla

›

Las Palmeras
Avenida del Parque 4, Sevilla

›

Residencial El Palmar
Calle Naranjos 8, Dos Hermanas

›

Comunidad San Juan
Plaza Mayor 1, Coria del Río

›

Urbanización Los Pinos
Calle Pinos 22, Alcalá de Guadaira

›

Continuar

- Login:

Convive

Accede a tu comunidad

Nº DE PISO

Ej: 2B

Planta y puerta de tu vivienda

CONTRASEÑA

••••••••

Piso o contraseña incorrectos.

Entrar

¿No tienes cuenta? [Regístrate](#)

- Registro:

Este recibe un código creado por el presidente asignado a una planta y piso concreto.

Convive

Crea tu cuenta de vecino

CÓDIGO DE INVITACIÓN

CÓDIGO

X X X X X

Tu presidente te ha proporcionado este código

DATOS PERSONALES

NOMBRE

Tu nombre

PRIMER APELLIDO

Apellido 1

SEGUNDO APELLIDO (OPCIONAL)

Apellido 2

CORREO ELECTRÓNICO

tu@correo.com

TELÉFONO

600 000 000

CONTRASEÑA

CONTRASEÑA

Mínimo 8 caracteres

REPETIR CONTRASEÑA

Crear cuenta

¿Ya tienes cuenta? [Inicia sesión](#)

- Home:

Bienvenido a Montepinar

Comunidad de propietarios · Calle Girasol 12, Sevilla

Hola, Alejandro · Piso 2B · Vecino

PRESUPUESTO 2026

24.000 €

9.450 € gastados (39%)

Ver presupuesto →

GASTOS ESTE MES

600 €

2 gastos registrados

Ver gastos →

VOTACIONES ABIERTAS

2

1 pendiente tu voto

Ver votaciones →

Votaciones recientes

Ver todas →

Renovación del ascensor — Bloque A

Cierra: 28 mar 2026 · 14/20 votos

Voto pendiente

Contratación servicio de limpieza

Cierra: 5 abr 2026 · 8/20 votos

Abierta

Últimos gastos

Ver todos →

Pintura zonas comunes

Marzo 2026 · VARIABLE

1.400,00 €

Seguro del edificio

Enero 2026 · FIJO

450,00 €/mes

- Usuarios:

Gestión de usuarios

Vecinos

Invitaciones

NOMBRE	PISO	CORREO	TELÉFONO	ROL	ESTADO	ACCIONES
María González	1A	maria@correo.com	600 111 222	Presidenta	Activo	Editar
Alejandro Peña	2B	alejandro@correo.com	600 333 444	Vecino	Activo	EditarDeshabilitar
Carmen Ruiz	3C	carmen@correo.com	600 555 666	Vecino	Inactivo	EditarHabilitar

Invitaciones

+ Generar código

A3K9X2

Piso: 4D
Expira: 24 mar 2026

Pendiente

Cancelar

B7M2P4

Piso: 2B
Usado: 10 mar 2026

Usado

X1Q5R8

Piso: 3C
Expiró: 1 mar 2026

Expirado

- Generación de invitación:

Generar código de invitación

PISO

Selecciona un piso

Solo se muestran pisos sin vecino activo

VALIDEZ DEL CÓDIGO

7 días

Cancelar

Generar código

- Pisos:

Gestión de pisos

+ Añadir piso

TOTAL PISOS	OCUPADOS	VACÍOS	INACTIVOS
12	8	3	1

Estado: Todos

Planta: Todas

PLANTA	PUERTA	VECINO ASIGNADO	ESTADO	ACCIONES
1	A	María González	Ocupado	EditarDesactivar
1	B	—	Vacio	EditarGenerar invitaciónDesactivar
2	A	Alejandro Peña	Ocupado	EditarDesactivar
2	B	Carmen Ruiz	Ocupado	EditarDesactivar
3	A	—	Vacio	EditarGenerar invitaciónDesactivar
3	B	—	Inactivo	EditarActivar

- Presupuesto:

Presupuesto actual

Abierto

Editar

+ Añadir gasto

Comunidad Las Palmeras · 1 ene 2026 → 31 dic 2026

← Ver presupuestos anteriores

IMPORTE TOTAL

24.000,00 €

GASTADO

9.450,00 €

39% del total

DISPONIBLE

14.550,00 €

Ejecución del presupuesto

39% ejecutado

CONCEPTO	TIPO	MES	COSTE	
Gastos fijos				
Seguro del edificio	FIXED	Ene	450,00 €/mes	Editar
Mantenimiento ascensor	FIXED	Ene	150,00 €/mes	Editar
Gastos variables				
Reparación gotera portal	VARIABLE	Feb	850,00 €	Editar
Pintura zonas comunes	VARIABLE	Mar	1.400,00 €	Editar

FONDO DE EMERGENCIA

3.200,00 €

Editar fondo

Reserva para imprevistos del ejercicio

Cerrar presupuesto

Una vez cerrado no podrá editarse. Se creará el nuevo presupuesto a continuación.

Cerrar presupuesto

- Historial de presupuestos:

← Volver al presupuesto actual

Historial de presupuestos

Presupuesto 2026

1 ene 2026 → 31 dic 2026

En curso

24.000,00 €

9.450,00 € gastados

Abierto

Ver

Presupuesto 2025

1 ene 2025 → 31 dic 2025

Cerrado el 15 ene 2026

22.000,00 €

21.340,00 € gastados

Cerrado

Ver

Presupuesto 2024

1 ene 2024 → 31 dic 2024

Cerrado el 10 ene 2025

20.500,00 €

19.800,00 € gastados

Cerrado

Ver

- Gastos:

Gastos

+ Añadir gasto

PRESUPUESTO TOTAL

24.000,00 €

1 ene 2026 → 31 dic 2026

TOTAL GASTADO

9.450,00 €

39% del presupuesto

ESTE MES

1.400,00 €

Marzo 2026

Mes:

Marzo

 | Tipo:

Todos

CONCEPTO	TIPO	MES	COSTE	ACCIONES
Marzo				
Seguro del edificio	FIXED	Mar	450,00 €	Editar Eliminar
Pintura zonas comunes	VARIABLE	Mar	1.400,00 €	Editar Eliminar
Total mostrado			1.850,00 €	

- Votaciones:

Votaciones

+ Nueva votación

Renovación del ascensor — Bloque A

Cierra: 28 mar 2026 14 / 20 votos Pendiente tu voto

Abierta

+ Info

Contratación servicio de limpieza

Cierra: 5 abr 2026 8 / 20 votos Ya votaste

Abierta

+ Info

Aprobación presupuesto anual 2026

Cerrada: 1 mar 2026 18 / 20 votos Aprobada

Cerrada

+ Info

- Páginas que no existen o accesos no autorizados:

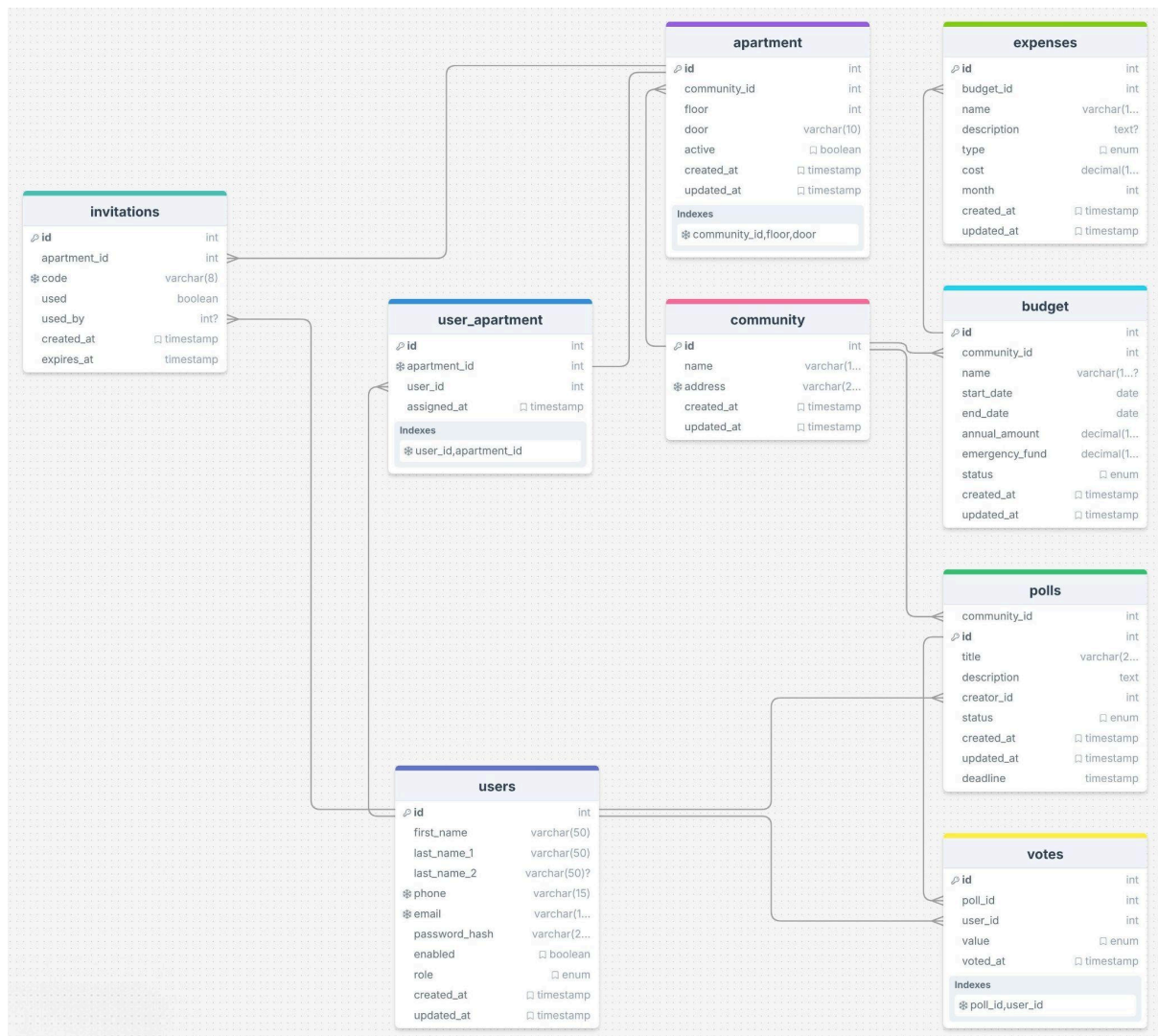


Base de datos:

Para la base de datos elegimos PostgreSQL porque soporta CHECK constraints directamente en las tablas, lo que nos permite validar datos a nivel de base de datos (por ejemplo, que el rol sea PRESIDENT o RESIDENT, que el importe de un presupuesto sea positivo, o que la fecha de fin sea posterior a la de inicio). Con MySQL estas validaciones habría que hacerlas solo en el backend, perdiendo una capa de seguridad.

La base de datos se ejecuta dentro de un contenedor Docker para que cualquier persona que descargue el proyecto pueda levantarlo con un solo comando (docker compose up -d) sin tener que instalar PostgreSQL en su máquina, configurar usuarios ni preocuparse por versiones. Esto garantiza que el entorno es idéntico en cualquier ordenador, ya sea Linux, Windows o Mac.

Aquí tenemos el esquema:



En la carpeta database/init/ incluimos dos archivos SQL que Docker ejecuta automáticamente al crear el contenedor por primera vez:

“01_schema.sql” crea todas las tablas con sus relaciones y restricciones y “02_seed.sql” inserta datos de prueba para que la aplicación no esté vacía al inicio.

Manual de uso:

- Primero clonamos el repositorio:

```
git clone https://github.com/05alejandro27/convive.git
```

- Agregamos las variables de entorno:

Hay que editar el .env.example añadiendo estos valores:

DB_NAME=convive

DB_USER=convive_user

DB_PASSWORD=convive_pass

DB_PORT=5433

Y cambiarle el nombre de .env.example a .env

- Levantar la base de datos:

En la carpeta “/convive/database/init” usamos el siguiente comando:

`docker compose up -d`

Y podemos comprobar el funcionamiento con este comando:

`docker ps`

- Arrancar el backend:

Dentro de tu IDE de confianza levantas el servicio o desde la carpeta “/convive/backend” lanzas el siguiente comando:

`mvn spring-boot:run`

- Arrancar el frontend:

Dentro de tu IDE de confianza levantas el servicio o desde la carpeta “/convive/frontend” lanzas los siguientes comandos:

`npm install`

`npm run dev`

- Abrir la aplicación web:

La aplicación se iniciará en “<http://localhost:5173>” por lo que desde tu navegador debes acceder a esta URL

- Uso:

Para acceder los datos de prueba son los siguientes:

Planta: 1

Puerta: 2

Contraseña: password123

- Borrar la base de datos:

En la carpeta “/convive/database/init” usamos el siguiente comando:

`docker compose down -v`

Lecciones aprendidas:

- El hash de contraseñas en los datos iniciales

Al insertar usuarios en el archivo 02_seed.sql, las contraseñas estaban hasheadas con BCrypt para una versión anterior de Spring Boot. Al actualizar a Spring Boot 4, no reconocía los hashes antiguos como válidos, por lo que el login fallaba con un fallo de: "credenciales incorrectas" sin ningún mensaje que indicará que el problema estaba en el hash. Fue difícil de encontrar porque el error no estaba en la lógica ni en el código, sino en la incompatibilidad entre la versión del hash almacenado y la versión de BCrypt que usaba Spring Security. La solución fue regenerar los hashes con la versión actual y sustituirlos en los datos iniciales.

- Records en Java para DTOs

Descubrí que Java 16 introdujo los record, que son perfectos para DTOs porque eliminan todo el código repetitivo. Los record generan automáticamente los campos como private final, el constructor, los métodos de acceso (".floor()" en vez de ".getFloor()"), y los métodos "equals()", "hashCode()" y "toString()". Esto hace que los DTOs sean inmutables por diseño, lo cual tiene sentido porque un request de login no debería poder modificarse después de crearse.

- JwtService manual vs Spring Security OAuth2 Resource Server

Spring ofrece una solución más completa con OAuth2 Resource Server, que permite integrar proveedores de identidad externos como Google o Keycloak. Decidí implementar un JwtService propio por dos razones:

Primero, porque quería entender cómo funciona JWT por dentro (generación, firma, claims), algo que OAuth2 Resource Server hace automáticamente.

Segundo, porque para mi MVP con un solo método de autenticación (planta + puerta + contraseña), un JwtService propio es más sencillo y suficiente.

- Seguridad con Boolean.FALSE.equals() para evitar NullPointerException

En el AuthService, al comprobar si un usuario está habilitado, use "Boolean.FALSE.equals(user.getEnabled())" en vez de "!user.getEnabled()". Esto lo

hice porque "getEnabled()" devuelve un Boolean (objeto), no un boolean (primitivo), y podría ser nulo. Si el campo es nulo y escribimos "!user.getEnabled()", Java intenta hacer unboxing del null a boolean y lanza un NullPointerException. Con Boolean.FALSE.equals() los tres casos están cubiertos: si es false entra en el if, si es true no entra y si es null tampoco entra ni lanza error.

Esto es algo que ya vi pero recordar al hacer el AuthService y me parecía interesante apuntarlo para no olvidarme más.

- Separación de DTOs en request y response

Con esta separación, el LoginRequest solo tiene los campos que envía el frontend (planta, puerta, contraseña) y el LoginResponse sólo devuelve lo necesario (el token). Cada DTO tiene una responsabilidad clara.

Estos ejemplos son para el Login pero en el futuro se usarán para otros DTOs

- Mala estimación de tiempos y replanificación

La planificación inicial tenía sprints con tareas demasiado genéricas ("diseñar frontend", "crear funcionalidades mínimas"...), sin concretar qué funcionalidades que se iban a desarrollar en cada uno. Al ejecutar los dos primeros sprints me di cuenta de que los tiempos estimados no se correspondían con el trabajo real, algunas tareas llevaban más tiempo del previsto y otras menos. Esto me ha hecho rehacer la planificación completa, reorganizando los sprints por funcionalidad concreta (pisos, usuarios, invitaciones...) en vez de por tareas técnicas abstractas. También creo que no le dedique el tiempo suficiente por subestimar la complejidad del proyecto, por lo que a partir de ahora me comprometeré a dedicar más horas semanales al proyecto para cumplir con los hitos establecidos.